

VIJAY PUTTAREVAIAH

Halifax, NS | +1 902-452-2085 | vijayputtarevaiah@gmail.com | vijay.p@dal.ca | [LinkedIn](#) | [GitHub](#)

Eligible to work in Canada

Software engineer with 5 years of professional experience spanning production software engineering at Wipro and Acuver Consulting Private Limited and fraud-risk analysis at Amazon, most recently architecting a vendor-failover system for Acuver client BJ's Wholesale Club that cut its vendor-caused order-cancellation rate from 10% to under 1%. Now completing a Master of Applied Computer Science at Dalhousie University, with a focus on system design, distributed architecture, and full-stack development spanning Java, Spring Boot, Microservices, React, and Next.js.

EDUCATION

Dalhousie University | GPA: 3.88 / 4.30

Halifax, NS

Master of Applied Computer Science (MACS) Jan 2026 – Apr 2027 Coursework: Advanced Topics in Software Development, Database Management & Warehousing Analytics, Advanced Topics in Web Development, Advanced Cloud Architecture

National Institute of Engineering, Mysuru (VTU)

India

Bachelor of Engineering in Electrical & Electronics Engineering (EEE)

Aug 2015 – Jul 2019

WORK EXPERIENCE

Software Engineer

Jan 2025 – Dec 2025

Acuver Consulting Private Limited

Bengaluru, India

Smart Order Rerouting + DoorDash Full Service | Client: [BJ's Wholesale Club](#)

Jun 2025 – Dec 2025

Technologies: Java, Spring Boot, Spring Security, JWT, Microservices, AWS, DynamoDB, Docker, Kubernetes

- Designed and built **Smart Order Rerouting**, automatically rerouting same-day delivery orders between **DoorDash** and **Roadie** when a vendor initiated a cancellation, cutting BJ's vendor-caused order-cancellation rate from ~10% to under 1% of total yearly online order volume
- Built configurable kill-switch logic to enable rerouting on a per-club basis, giving the team controlled, club-by-club rollouts and a fast way to disable the feature at a specific location the moment an issue surfaced
- Led the **DoorDash Full Service** project, integrating **DoorDash's Serviceability APIs** to validate delivery coverage and automating the reservation-creation workflow, letting DoorDash agents pick, pack, and deliver orders directly from BJ's club locations
- Hardened third-party API communication with error handling, fallback paths, and retry logic to keep the rerouting pipeline reliable against **DoorDash** and **Roadie** API failures
- Documented the complete **VendorHub** backend, the **Spring Boot** application Smart Order Rerouting was built in, through **JavaDocs**, technical documentation, and **Postman** API collections for the engineering team
- Secured Smart Order Rerouting's internal APIs with **Spring Security** and **JWT**-based authentication, restricting access to authorized internal services only

Carrier Management Services (CMS) | Client: [Titan](#)

Mar 2025 – May 2025

Technologies: Java, Spring Boot, Spring WebFlux, Spring Data JPA, MongoDB, React.js

- Built and integrated **Turnaround Time (TAT)** APIs with **Spring Boot** and **Spring WebFlux**, applying the **Strategy pattern** to swap per-carrier integration logic, polling **BlueDart's** and **Criticallog's** REST APIs on a scheduled cron interval since neither carrier supported webhooks, replacing manually-tracked delivery windows with automated, carrier-sourced data
- Built a webhook-driven integration with carrier **Delhivery**, the one partner offering webhook support, to receive real-time shipment-tracking and carrier-serviceability updates without polling
- Automated shipment creation by building Waybill Generation APIs that mapped internal order data into each carrier's request format and invoked their external APIs directly, cutting manual data entry from the fulfillment workflow
- Added fallback handling for carrier-specific AWB (airway bill) generation failures, keeping shipment creation resilient when an individual carrier's API was unavailable
- Calculated estimated delivery dates by layering business-rule buffers on top of measured TAT, giving the client's ops team a consistent EDD figure per shipment
- Built a **React.js** frontend for the CMS giving ops visibility into live package tracking, designed so a new carrier partner could be added or removed through the UI without any backend code change

*[Aekyam 2.0](#) | Acuver's own **iPaaS** product*

Jan 2025 – Feb 2025

Technologies: Java, Spring Boot, Microservices, Spring WebFlux, PostgreSQL, React.js, Redux

- Resolved bugs across the **Platform Scheduler** and **Platform Common** libraries, and shipped features and fixes across the **Platform Workflow**, **Platform Inbound**, and **Platform Scheduler** microservices

- Built a scoped-bypass private API endpoint in Platform Inbound for designated internal system processes, an automatic retry path for failed-entry transactions matching specific error codes, and a REST PATCH endpoint for subscribed customer data in the platform marketplace, cutting unnecessary data transfer on updates
- Built a **Platform Workflow** to modify and transform incoming **Order Management System (OMS)** requests into the format required by downstream systems, including **Titan's Carrier Management System (CMS)**
- Built **React.js** components for Aekyam's frontend, using **Redux** and **React Hooks** for state management, including pagination in the **Audit Service UI**

Software Engineer

Wipro Limited

Apr 2022 – Jul 2024

Bengaluru, India

Integrated Scrubbing Modernization | Client: [Oracle Cerner Corporation](#)

May 2022 – Feb 2024

Technologies: Java, Spring Boot, OAuth 2.0, Microservices, MySQL, Jenkins, Docker, Kubernetes, Edifecs

- Built a **Spring Boot** health-check service from scratch, monitoring both **Dockerized** and non-Dockerized services and servers across the organization to catch failures before they reached end users
- Cut infrastructure costs **20%** by migrating real-time claims scrubbing from a legacy **IBM BPM** platform to **Edifecs**, refactoring the scrubbing profiles and **containerizing** the resulting Spring Boot applications and Edifecs profiles with **Docker**
- Led deployment of the modernized platform onto **Kubernetes** across multiple environments via **Jenkins** pipelines, resolving deployment issues and coordinating release activities to keep rollouts on schedule
- Secured integration endpoints between the modernized platform and **Edifecs** with **OAuth 2.0**, ensuring only authorized services could invoke claims-scrubbing operations
- Resolved production bugs surfaced after each release, working with the operations team to root-cause and fix issues in the live claims-scrubbing environment, coordinating deployments across **Development**, **UAT**, and **Production**

MTS | Client: [Standard Chartered Bank](#)

Feb 2024 – Jul 2024

Technologies: Java, Spring Boot, MySQL

- Built REST APIs with **Spring Boot** and **MySQL** to monitor message queue (MQ) depth and failed-message counts in real time
- Implemented a threshold-triggered verification flow that automatically flagged failed messages once the MQ failure count crossed a set limit, speeding up root-cause analysis for the operations team
- Coordinated release and deployment activities for the engagement, working across environments to keep the messaging-monitoring service running through each rollout

Transaction Risk Investigator

Amazon

Sep 2020 – Apr 2022

Bengaluru, India

Project: A to Z Claims

Sep 2020 – Apr 2022

Technologies: Java, SQL, Paragon

- Analyzed transaction data with **SQL**, surfacing patterns in fraudulent activity that sharpened the accuracy of downstream fraud-detection decisions
- Implemented new detection-tool features in **Java** and tuned the performance of the existing fraud-detection pipeline, contributing to a **30% increase in fraud-detection accuracy**
- Completed Amazon's **ATLAS** program, an internal software-engineering upskilling track, building REST operations and resolving software bugs as part of the curriculum
- Partnered with stakeholders to update Standard Operating Procedures (SOPs), keeping investigation documentation aligned with current process and policy

PROJECTS

MeetFocus – Video Conferencing Platform | [GitHub](#)

2026

Technologies: Java · Spring Boot · WebSocket · Python · FastAPI · Kafka · Docker

- Designed and built a link-less video conferencing platform across 5 **Spring Boot** microservices (user, meeting, signaling, gateway, discovery) plus a Python **FastAPI** moderation service, coordinated via **Kafka** and Eureka service discovery
- Built real-time meeting-invite delivery over **WebSocket** sessions in the signaling service, relaying Kafka-published invite events straight to an already-connected attendee's open session
- Built a thread-safe focus-telemetry aggregation engine for real-time meeting-attention tracking, validated with concurrency tests simulating 150 parallel attendee connections
- Implemented a chat-moderation pipeline routing messages through **Kafka** to a Python toxicity-classification service and back, blocking flagged messages before broadcast
- Containerized the full system (5 Java services, 1 Python service, PostgreSQL, Redis, Kafka) with **Docker Compose** for local orchestration

Technologies: AWS Bedrock · Python · Step Functions · Terraform · Java Spring Boot · Docker

- Built a solo **autonomous incident-response control plane** that detects, diagnoses, and remediates production incidents on a **Java Spring Boot** fintech workload, with a deterministic policy engine kept outside the LLM's reasoning loop and a human-approval gate for high-risk actions
- Orchestrated **10 Python Lambda functions** through an **AWS Step Functions** state machine (intake, evidence, diagnose, policy, remediate, verify, postmortem, approval), triggered automatically by **CloudWatch** alarms via **EventBridge**
- Integrated **AWS Bedrock** (Nova Lite) to diagnose incidents from **CloudWatch Logs Insights** and metrics into structured JSON, feeding a separate **DynamoDB**-backed policy engine that decides auto-remediation versus human approval, so the model only ever names an action and never calls AWS APIs directly
- Routed high-risk remediation actions through an approve/reject workflow using Step Functions' `waitForTaskToken`, **SNS**, and **API Gateway**, resuming the paused execution the moment a human responds
- Provisioned the full agent and workload plane as **Terraform** infrastructure-as-code across **~100 managed resources** (16 least-privilege **IAM** roles, 10 Lambdas, 3 **DynamoDB** tables, full **ECS/ALB/ECR** stack), and stood up **GitHub Actions** CI/CD authenticating via **OIDC federation** with no long-lived AWS credentials stored anywhere
- Engineered the pipeline to a **sub-120-second MTTR** target and **sub-60-second** alarm-detection latency on the auto-approved remediation path, validated end-to-end with built-in chaos-engineering endpoints (error storms, latency injection, task crashes)
- Unit-tested the agent plane's deduplication and policy-engine logic with **pytest**, mocking AWS services via **moto** so the full suite runs without any real AWS calls or cost

LEDGR – Personal Finance & Bill-Splitting App | [GitHub](#)

2026

Technologies: Next.js · React Native · Supabase · TypeScript

- Designed and built **LEDGR**, a full-stack personal finance and bill-splitting app on **Next.js** and **Supabase** (Postgres, Auth, Storage, Row-Level Security), where a user's share of any shared group expense flows automatically into their personal ledger and analytics with no manual reconciliation
- Architected an AI feature kill-switch for categorization, receipt OCR, and analytics narrative generation (**Anthropic Claude**, **Google Cloud Vision**) with defense-in-depth gating: every route independently re-verifies server-side regardless of client state, and AI UI elements are absent rather than hidden when a feature is off
- Hardened the app through real security testing: a **Semgrep** SAST pass returned zero findings and an **OWASP ZAP** DAST baseline scan's findings were resolved via security headers, layered on **RLS** enforced at the database and **zod** validation as the server-side authorization boundary on every mutating route
- Built a **63-test** suite (51 unit, including property-based fuzzing of the split-math reconciliation invariant across 200 randomized runs; 10 integration covering cross-user RLS isolation; 2 end-to-end persona scenarios) running in **CI** on every pull request
- Grounded the product in real UX research, translating two named user personas and a documented heuristic evaluation into the core auto-flow mechanic and specific interface fixes
- Measured and fixed **5 performance bottlenecks** (3 client-side, 2 server-side), documenting a before/after number and reproduction method for each
- Built two idempotent, bearer-token-gated cron endpoints handling daily recurring-transaction generation and monthly budget rollover automatically
- Built out **22+ REST API routes** with **Upstash Redis**-backed rate limiting that falls back gracefully to an in-memory limiter when Redis is unavailable
- Built a **React Native** mobile app covering **iOS** and **Android**, mirroring the same core finance-tracking features and REST API integration as the **Next.js** web app
- Tested the web app across major browsers for cross-browser compatibility

Recalibrate – AI-Assisted Adaptive Goal-Tracking Platform | [GitHub](#)

2026

Technologies: Python · FastAPI · React · MySQL · AWS EC2 · Docker

- Collaborated in a **4-person team** to build Recalibrate, a full-stack adaptive goal-tracking platform, taking the lead across CI/CD, code quality, and core product features as the team's largest individual contributor
- Owned the **CI/CD pipeline** end-to-end on **GitLab CI** (lint, test, Docker build) through deployment to **AWS EC2**, including the CORS and environment-variable fixes needed to connect the deployed frontend to the deployed backend correctly
- Versioned the **MySQL** schema through **Alembic** migrations, keeping database changes reproducible and rollback-safe across environments
- Drove the team's code-quality bar by integrating **DPy static design-smell detection** into the CI/CD pipeline as an enforced gate, then personally resolving flagged architectural smells in production code (e.g., a Feature Concentration smell in the core module)

- Enforced **SOLID** design principles (DRY, single-responsibility) across the backend and frontend through targeted refactors, holding the codebase to a standard beyond what the assignment required
- Architected and built the **LLM-powered goal-category detection and task-generation** feature behind a provider-agnostic interface (configurable across OpenAI and Google GenAI) rather than a hardcoded integration
- Built authentication flows (email verification with rate-limited resend, password reset, secure login) through disciplined **TDD**, a **cron-scheduled** deadline-notification system, and containerized the full stack with **Docker Compose** for reproducible local environments
- Built adaptive replanning: detects missed tasks against a goal's daily roadmap and automatically regenerates an adjusted task plan, with pause/resume support offering a keep-original-deadline or set-new-deadline regeneration path

Open-Source Contribution: Shopizer – E-Commerce Platform | [Pull Request](#)

2026

Technologies: Java · Refactoring · Design Patterns

- Submitted a refactoring pull request to **Shopizer**, an established open-source Java e-commerce platform, applying **8 refactoring techniques** without changing external behavior
- Replaced an extensive if-else chain in payment processing with a **strategy pattern**, extracted a dedicated shipping facade to separate concerns, and consolidated duplicate logic across mapper classes into shared utilities
- Passed the project's automated quality gate with zero new issues and zero added duplication, with the full existing test suite green; PR submitted and open for review

TECHNICAL SKILLS

Languages: Java, Python, TypeScript, JavaScript, SQL, HTML, CSS

Frameworks & APIs: Spring Boot, Spring MVC, Spring WebFlux, Spring Microservices, Spring Cloud, Spring Data JPA, Hibernate ORM, FastAPI, Django, REST APIs, SOAP, GraphQL, React.js, Next.js, Vite

Databases: PostgreSQL, MySQL, MongoDB, DynamoDB, H2

Cloud & DevOps: AWS (Bedrock, Lambda, Step Functions, ECS Fargate, EC2, ECR, DynamoDB, S3, SNS, SQS, API Gateway, EventBridge, CloudWatch, IAM, VPC, Security Groups), Supabase, Terraform, Docker, Kubernetes, Kafka, Redis (caching), Jenkins, GitHub Actions (OIDC), GitLab CI, CI/CD

Testing & Security: JUnit, Mockito, Vitest, TDD, Spring Security, JWT, OAuth 2.0, Semgrep (SAST), OWASP ZAP (DAST)

Tools: Git, GitHub, GitLab, Bitbucket, SVN, Postman, Jira, SOAP UI, Confluence, Putty, mRemoteNG, MS Office (Word/Outlook/Excel/PowerPoint)

Practices: Agile/Scrum, Waterfall, Design Patterns, SOLID principles, JavaDocs, Technical Documentation, Code Reviews

ACHIEVEMENTS, CERTIFICATIONS & LEADERSHIP

Awards: Habitat Flagbearer Award, Wipro Limited | Winner's Circle Award, Wipro Limited | Full Tuition Fee Scholarship (e-PASS), Government of Karnataka – all 4 years

Certifications: AWS Certified AI Practitioner, AWS Certified Cloud Practitioner [TODO: add verification links, 10 days out], AWS Certified Solutions Architect – Associate (Ongoing)

Leadership: Student Representative, Dalhousie Machine Learning Society – authored end-to-end planning documentation for a Machine Learning Hackathon to give students hands-on ML experience, and partnered with other student societies to organize supporting workshops